GigShield — Parametric Income Insurance for E-Commerce Delivery Partners

Guidewire DEVTrails 2026 | Phase 1 Submission

1. The Problem

Amazon and Flipkart delivery partners are the last mile of India's e-commerce infrastructure. They operate in unpredictable outdoor environments — extreme heat, heavy rain, floods, dense smog, curfews — with no income protection when disruptions force them off the road.

A Swiggy or Zomato worker has platform-level visibility. An Amazon flex partner has nothing. When a red-alert weather day wipes out their shift, they absorb 100% of the loss.

GigShield insures the income lost during these events. Nothing else.

> Coverage scope: LOSS OF INCOME ONLY.
> Explicitly excluded: vehicle repairs, health insurance, accident medical bills.

---

2. Persona & Scenario Walkthrough

Persona: Ravi, 27. Amazon Flex delivery partner, Pune.
Earns ₹600–800/day. Works 6 days/week. Supports a family of 4.

Scenario A — Extreme Heat Day

IMD issues a heat alert: temperature exceeds 43°C in Ravi's zone. Amazon Flex suspends outdoor deliveries. Ravi loses a full day's earnings.

GigShield Response:

- Weather API detects trigger threshold breach in Ravi's registered zone
- Policy coverage activates automatically — no claim needed
- ₹500 payout (mapped to his declared daily income tier) hits his UPI in under 2 minutes

Scenario B — Flash Flood / Red Alert Rain

Heavy rainfall causes waterlogging across Ravi's delivery zone. Deliveries suspended for 6 hours.

GigShield Response:

- Rainfall + flood-zone API confirms disruption
- Partial payout issued (proportional to hours affected)
- Fraud layer cross-checks Ravi's GPS zone against alert coverage area

Scenario C — AQI Spike (Severe Pollution)

AQI crosses 400 (Hazardous) in Ravi's city. Outdoor movement advisory issued.

GigShield Response:

- AQI API (CPCB / OpenWeatherMap) confirms trigger
- Policy activates; payout processed automatically

Scenario D — Local Curfew / Strike

An unplanned curfew is issued in Ravi's zone. Amazon suspends operations.

GigShield Response:

- Admin manually flags zone disruption via dashboard
- Workers in zone automatically receive coverage notification

- Claims processed in batch

3. Application Workflow

```
[Onboarding]
  Worker registers → KYC via Aadhaar OTP → Selects platform (Amazon/Flipkart)
  → Declares income tier → Chooses weekly plan → Pays weekly premium (UPI/Wallet)
  → Policy active

[Active Coverage]
  System polls weather/AQI/traffic APIs every 15 minutes per zone
  → Trigger threshold breach detected
  → Affected workers identified by registered zone
  → Automatic claim initiated (parametric — no manual filing)

[Payout]
  Fraud layer evaluates claim
  → Green Route: Instant UPI payout
  → Yellow Route: Step-Up verification (photo/video) → payout on pass
  → Red Route: Quarantined for manual review

[Dashboard]
  Worker: Active policy, payout history, weekly coverage status
  Admin: Zone alerts, loss ratios, fraud flags, predictive risk heatmaps
```

4. Weekly Premium Model

Gig workers operate week-to-week. Monthly insurance models don't fit their cash flow. GigShield is priced and settled **weekly**.

Pricing Tiers

Plan	Weekly Premium	Max Weekly Payout	Coverage
Basic Shield	₹29	₹500	1 disruption/week
Standard Shield	₹59	₹1,200	Up to 3 disruptions/week
Pro Shield	₹99	₹2,000	Unlimited disruptions/week

Dynamic Premium Calculation (AI Layer)

Base premium is adjusted weekly by a risk model using:

- **Zone risk score** — historical disruption frequency in worker's delivery zone
- **Seasonal modifier** — monsoon/summer surcharge; winter discount
- **Tenure discount** — loyal workers with clean claim history get lower premiums
- **Platform tier** — Amazon Flex Prime partners may qualify for subsidized rates

Example: A worker in a historically flood-prone zone in Mumbai during monsoon season pays ~15% above base. A worker in a low-risk Pune zone in winter pays ~10% below base.

5. Parametric Triggers

Triggers are automated, threshold-based, and require zero manual filing from the worker.

Trigger	Data Source	Threshold	Payout Type
Extreme Heat	IMD / OpenWeatherMap	Temp > 42°C in zone	Full day
Heavy Rain	OpenWeatherMap / IMD	Rainfall > 50mm/hr	Proportional
Flood Alert	NDMA / IMD red alert	Zone flood advisory issued	Full day

Trigger	Data Source	Threshold	Payout Type
Severe AQI	CPCB / OpenWeatherMap	AQI > 300 (Very Poor)	Full day
Curfew / Strike	Admin-flagged	Zone manually marked	Full day

All thresholds are zone-specific, not city-wide averages. A flood in Dharavi does not trigger coverage for a worker in Bandra.

---

6. Platform Choice: Web Application (Next.js)

Decision: Web platform, not native mobile.

Reasoning:

- GigShield serves two distinct user types — delivery workers and insurance admins — both well-served by a responsive web app
- A web platform allows richer admin dashboards (zone heatmaps, loss ratios, fraud analytics) that would be constrained on mobile
- No app store approval delays — workers onboard via a browser link, zero friction
- Next.js enables server-side rendering for fast initial load on low-bandwidth connections common in delivery zones
- Single codebase, responsive design — works on desktop for admins and on a phone browser for workers
- Faster to build, test, and demo within a 6-week hackathon timeline

---

7. AI/ML Integration Plan

Premium Calculation

- **Model:** Gradient Boosted Regression (XGBoost)
- **Features:** Zone historical disruption frequency, season, worker tenure, claim history, platform type
- **Output:** Dynamic weekly premium multiplier per worker
- **Training data:** IMD historical weather data + simulated claim history for Phase 1

Fraud Detection — Adversarial Defense & Anti-Spoofing Layer

> *Reference:* `/docs/adversarial-defense.md`

The threat: Coordinated GPS-spoofing syndicates (organized via Telegram) fake their location inside red-alert weather zones to mass-file false claims. Simple GPS verification is trivially defeatable. This is what replaces it.

Spatial-Temporal Anomaly Detection Pipeline

Cluster Analysis — Syndicate Detection

Coordinated fraud requires coordination. Coordination produces detectable patterns.

```
Weather Alert Goes Live
  -> Claims spike from tight geographic radius
  -> DBSCAN clustering detects density anomaly
  -> Entire cluster flagged for review
```

Threshold: 15+ claims within 500m radius in under 60 seconds = automatic escalation.

Behavioral Velocity Model

```
Legitimate worker  ->  stops -> assesses -> triggers claim  (minutes)
Fraudster at home  ->  triggers claim the instant alert fires  (seconds)
```

The model tracks reaction time against zone-specific baselines trained on historical legitimate claims. Sub-minute reaction = flagged.

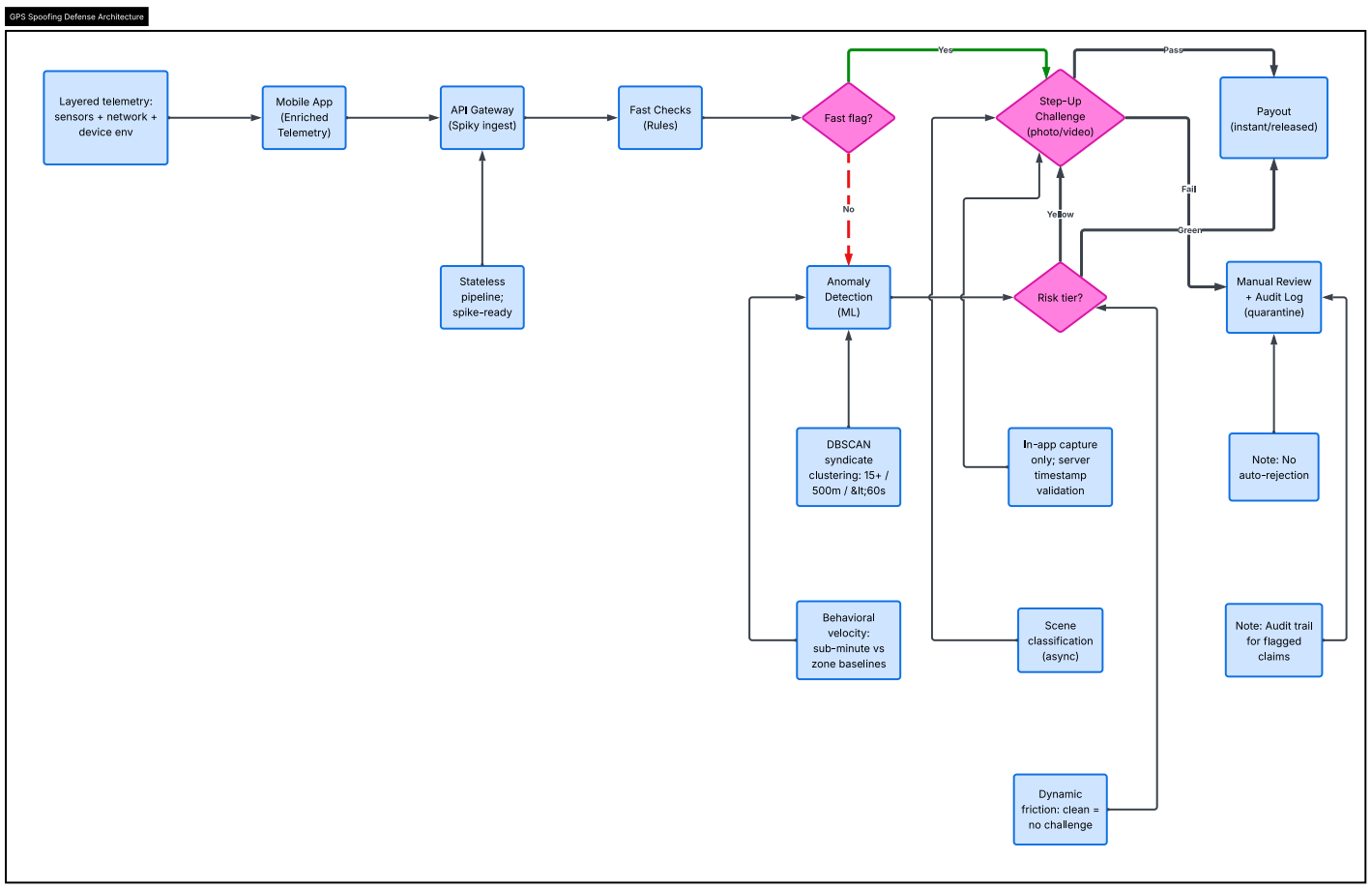
Multi-Layer Telemetry Stack

GPS coordinates are trivially spoofable. The system ingests a full telemetry stack:

Signal	Data Points	What It Catches
Sensor Telemetry	Accelerometer, gyroscope	Static device (coffee table) vs. dynamic (storm)
Network Infrastructure	IP address, cell tower ID, Wi-Fi BSSID	GPS says Zone A, carrier resolves 50km away
Device Environment	OS mock location flag, VPN usage, emulator signature	Developer spoofing tools, known GPS apps
Cluster Metadata	Timestamps, geographic density, concurrency	Coordinated group behavior

Defeating any single signal still leaves multiple corroborating anomalies.

System Architecture Diagram



> *Full pipeline: Layered telemetry → Fast Path → ML Pipeline → Green/Yellow Route → Dynamic Friction Protocol*

Two-Stage Pipeline Architecture

- **Stage 1 — Fast Path:** Redis-backed Static Rule Engine checks mock location flags, VPN IPs, and emulator environments in milliseconds. If flagged, bypasses ML entirely → Yellow Route.
- **Stage 2 — ML Pipeline:** Three parallel models on claims that pass Stage 1:
  - DBSCAN spatial clustering (density spikes)
  - Behavioral Velocity Model (reaction times vs. zone baselines)
  - Sensor Telemetry Classifier (static vs. dynamic device)

Pipeline resolves to a confidence score → **Green Route** (instant payout) or **Yellow Route** (Step-Up Challenge).

Dynamic Friction Protocol — UX Balance

Auto-rejection punishes honest workers. Genuine network instability during a real storm can produce false-positive anomalies.

```
Claim Flagged by ML Pipeline
  -> Payout temporarily paused
  -> Worker receives Step-Up Challenge (in-app)
  -> Must submit real-time photo or 3-second video
```

```
Pass -> Payout released immediately
Fail -> Claim quarantined for manual review
```

- **In-app capture only** — camera triggered natively, no gallery uploads
- **Timestamp validation** — capture time verified server-side against claim time
- **Scene classification** — async background task checks image against declared weather
- **Proportional friction** — clean telemetry = zero additional steps, ever
- No auto-rejections — flagged claims are always human-reviewable
- No hard bans on first offense — quarantine first, investigate second

✔️ Honest workers: take a 3-second video, get paid
✔️ Fraudsters: cannot produce real-time evidence of a storm they're not in
✔️ This architecture does not patch the vulnerability — it makes the attack vector unprofitable

Risk Heatmapping

- Zone-level predictive risk scores updated weekly
- Admin dashboard visualizes next-week likely disruption zones
- Used to dynamically adjust liquidity reserves

---

8. Tech Stack

Layer	Technology
Frontend	Next.js 14, Tailwind CSS
Auth Service	Node.js + Express (microservice — registration, KYC, JWT sessions)
ML Service	FastAPI (microservice — premium calculation, fraud detection, risk scoring)
Database	PostgreSQL (policies, claims, users) + Redis (fast-path fraud rules, caching)
ML Models	Python, scikit-learn, DBSCAN, XGBoost
Weather APIs	OpenWeatherMap (free tier), IMD public feeds, CPCB AQI API
Payments	Razorpay test mode (UPI mock payouts)
Hosting	Vercel (frontend) + AWS EC2 (Express + FastAPI services)
Messaging	FCM (disruption alerts to workers)

Architecture: Next.js frontend communicates with two independent microservices — Express handles all auth/session concerns, FastAPI owns the entire ML and claims pipeline. Both backed by shared PostgreSQL + Redis.

---

9. Development Plan

Phase 1 (March 4–20) — Ideation & Foundation

- [x] ~~Problem definition and persona research~~
- [x] ~~Weekly premium model designed~~
- [x] ~~Parametric triggers defined~~
- [x] ~~Fraud detection architecture designed (adversarial defense doc)~~
- [x] ~~Tech stack finalized~~
- [x] ~~GitHub repo initialized with README~~

Phase 2 (March 21 – April 4) — Automation & Protection

- [ ] Worker registration + KYC flow
- [ ] Policy creation with dynamic weekly premium calculation
- [ ] Parametric trigger engine (weather API polling per zone)

- [ ] Automated claim initiation pipeline
- [ ] Basic fraud detection (Stage 1 fast path)
- [ ] Mock UPI payout flow (Razorpay test mode)

Phase 3 (April 5–17) — Scale & Optimise

- [ ] Full ML fraud pipeline (DBSCAN + velocity model + sensor telemetry)
- [ ] Step-Up Challenge (in-app camera verification)
- [ ] Worker dashboard (earnings protected, active coverage, payout history)
- [ ] Admin dashboard (loss ratios, fraud flags, predictive risk heatmaps)
- [ ] Final 5-minute demo video
- [ ] Final pitch deck

10. Repository Structure

```
gigshield/
├── frontend/                          # Next.js 14 — worker app + admin dashboard
│   ├── app/
│   │   ├── (worker)/
│   │   │   ├── dashboard/
│   │   │   │   └── page.tsx           # Worker dashboard — coverage, payouts
│   │   │   ├── onboarding/
│   │   │   │   └── page.tsx           # Registration + KYC flow
│   │   │   └── policy/
│   │   │       └── page.tsx           # Weekly plan selection + payment
│   │   ├── (admin)/
│   │   │   ├── dashboard/
│   │   │   │   └── page.tsx           # Admin — loss ratios, fraud flags
│   │   │   └── zones/
│   │   │       └── page.tsx           # Zone heatmap + manual curfew flag
│   │   ├── layout.tsx
│   │   └── page.tsx                   # Landing / login
│   ├── components/
│   │   ├── ui/                        # Shared UI components
│   │   ├── PolicyCard.tsx
│   │   ├── PayoutStatus.tsx
│   │   └── ZoneHeatmap.tsx
│   └── lib/
│       ├── api.ts                     # API client (auth + ml service calls)
│       └── constants.ts
│
├── auth-service/                      # Express — registration, login, JWT, KYC
│   ├── src/
│   │   ├── api/
│   │   │   ├── auth/
│   │   │   │   ├── auth.routes.ts
│   │   │   │   └── auth.controller.ts
│   │   │   └── worker/
│   │   │       ├── worker.routes.ts
│   │   │       └── worker.controller.ts
│   │   ├── middlewares/
│   │   │   ├── jwt.middleware.ts
│   │   │   └── error.middleware.ts
│   │   ├── core/
│   │   │   ├── startup.ts
```

```
│   │   │   └── shutdown.ts
│   │   └── index.ts
│   └── package.json
│
├── ml-service/                        # FastAPI — ML pipeline + trigger engine
│   ├── app/
│   │   ├── api/
│   │   │   ├── premium/
│   │   │   │   ├── __init__.py
│   │   │   │   └── premium.py         # Dynamic weekly premium calculation
│   │   │   ├── claims/
│   │   │   │   ├── __init__.py
│   │   │   │   └── claims.py          # Parametric trigger + claim initiation
│   │   │   ├── fraud/
│   │   │   │   ├── __init__.py
│   │   │   │   └── fraud.py           # DBSCAN + velocity + telemetry pipeline
│   │   │   └── api_router.py
│   │   ├── controllers/
│   │   │   ├── __init__.py
│   │   │   ├── fraud_controller.py
│   │   │   └── premium_controller.py
│   │   ├── core/
│   │   │   ├── __init__.py
│   │   │   ├── startup.py
│   │   │   └── shutdown.py
│   │   ├── middlewares/
│   │   │   └── __init__.py
│   │   └── model/
│   │       ├── premium_model.pkl      # XGBoost premium calculator
│   │       ├── fraud_classifier.pkl   # Sensor telemetry classifier
│   │       └── train_models.ipynb     # Training notebook
│   └── requirements.txt
│
├── docs/
│   ├── adversarial-defense.md         # Full GPS spoofing defense spec
│   └── architecture.md               # System architecture diagrams
│
├── scripts/
│   ├── poll_weather.py                # OpenWeatherMap zone polling
│   ├── poll_aqi.py                    # CPCB AQI zone polling
│   └── simulate_trigger.py           # Manual disruption trigger simulation
│
└── README.md
```

---

11. Team

Aditya

Palak Maurya

Prathmesh Desai

GigShield — Because the last mile shouldn't be the first to bear the loss.